Study book

Topic 3: Interfaces, generics, exceptions

ProjectHub learning track: React/TypeScript to C#/.NET backend. Week 1, topic 3 of 20.

Brief overview

What this lesson is about. How C# separates a **contract** (an interface) from an **implementation** (a class), why code should depend on the contract, how to write your own **generics** (generic methods, constraints, generic types), and how to design **exceptions**: expected vs unexpected errors, custom exception types, `null` vs throwing, `throw;` vs `throw ex;`.

What we built. The first real backend layer: `IProjectService` + `ProjectService` (in-memory, with a business rule: unique project names), two exception types (`NotFoundException`, `BusinessRuleException`), and a 10-section demo. In Rider you used a **.NET exception breakpoint** to stop exactly where an exception is thrown.

Why it matters for the interview. 'Interface vs class / abstract class', 'why program against an interface', 'expected vs unexpected errors', 'throw vs throw ex', and 'why constructor injection' are standard .NET interview questions. Topics 6, 7, 10, 12 and 13 all build on this code.

Contents

Part	What is inside
1	Interfaces: contract vs implementation, nominal matching, why program against the interface, interface vs abstract class
2	Generics: generic methods and types, constraints, reified generics vs TS
3	Exceptions: try/catch/finally in C#, catch order, when filters, throw; vs throw ex;, expected vs unexpected, null vs throw
4	Our code: exceptions, IProjectService, ProjectService (with the constructor explained), design decisions, known limitations
5	`var` vs an explicit (interface) type: your question
6	The demo: 10 sections and full output
7	Debugging in Rider: .NET exception breakpoints, reading the Variables panel, unhandled exceptions
8	Your answers, corrections, and interview Q&A with model answers
9	Summary, cheat sheet, checklist, what comes next

Part 1. Interfaces

An interface is a **contract**: a list of members (methods, properties) **without implementation**. A class that implements it promises to provide all of them; if one is missing, the build fails.

```
public interface IProjectService
{
    Project? GetById(int id);
}

public class ProjectService : IProjectService   // ": IProjectService" = "I implement this contract"
{
    public Project? GetById(int id) => ...;     // must exist, or the build fails
}
```

1.1 The big difference from TypeScript

	TypeScript `interface`	**C#** `interface`
How a class matches	**structurally**: any object with the right shape fits	**nominally**: the class must **declare** `: IProjectService`; the compiler checks every member
Exists at runtime?	no, erased	yes: `service is IProjectService` works
Describes data shapes?	very often (`interface User { name: string }`)	rarely; data shapes are classes/records. Interfaces describe **behavior**
Naming	`User`, `ProjectService`	`I` prefix: `IProjectService`, `IEnumerable<T>`

1.2 Program against the interface

```
IProjectService service = new ProjectService(data);            // variable typed as the CONTRACT
static void PrintProjectCount(IProjectService service) { ... }  // works with ANY implementation
```

A variable typed as the interface can only use what the interface describes, even if the class has more public members (your own observation in the quiz). Benefits:

1. **Swap implementations.** Today an in-memory list; in topic 10 EF Core + SQL Server. Code that only knows `IProjectService` does not change.
2. **Testing (topics 13-14).** A test passes a fake implementation, so a controller can be tested without a database.
3. **Dependency injection (topic 7).** ASP.NET is configured to hand out `ProjectService` whenever someone asks for `IProjectService`.
4. **Decorators (demo section 7).** `LoggingProjectService` implements the same interface, wraps another implementation, adds logging; callers cannot tell the difference.

This is **polymorphism**: one interface, many implementations, chosen at runtime.

1.3 Interface vs abstract class vs class

	`interface`	`abstract class`	`class`
Implementation?	no (rare default methods aside)	partly: some members implemented, some `abstract`	fully
Fields / state?	no	yes	yes
Can you `new` it?	no	no	yes

	`interface`	`abstract class`	`class`
How many per class?	**many**: `class X : IA, IB, IC`	**one** base class	**one** base class
Use for	a contract or capability ('can do X')	shared base code for closely related types	the actual implementation

Modern .NET favors interfaces and **composition over inheritance**: `LoggingProjectService` *contains* another service instead of *inheriting* from one. The everyday exception to 'little inheritance': custom exceptions inherit from `Exception` (Part 4).

Part 2. Generics

You have used generics since topic 1 (`List<T>`, `Dictionary<TKey, TValue>`, `Func<T, bool>`). A generic has a **type parameter** `T` that the caller fills in, like TS generics.

```
// generic METHOD: T is inferred from the argument
static T Largest<T>(IEnumerable<T> items) where T : IComparable<T> { ... }
Largest(new[] { 3, 9, 4 });            // T = int     -> 9
Largest(new[] { "pear", "apple" });    // T = string  -> "pear"

// generic TYPE
record Page<T>(IReadOnlyList<T> Items, int PageNumber, int TotalCount);
Page<string> names = ...;   Page<TaskItem> tasks = ...;
```

2.1 Constraints: where T : ... (TS: T extends ...)

Without a constraint you can only do with `T` what works for every type. A constraint unlocks more:

Constraint	Meaning	Example
`where T : IComparable<T>`	T can be compared: `CompareTo` exists	`Largest<T>` in the demo
`where T : class`	reference type	makes `T?` mean 'nullable reference'
`where T : struct`	value type	`Nullable<T>` is declared like this
`where T : new()`	has a public parameterless constructor	`new T()` inside the method
`where T : BaseClass` / `IInterface`	is or implements it	use its members
`where T : notnull`	not nullable	dictionary keys

`Largest(service.GetAll())` does **not compile**: `Project` does not implement `IComparable<Project>`, so the constraint rejects it at compile time.

2.2 C# generics exist at runtime (unlike TS)

TS generics are **erased**: at runtime there is only `Array`. C# generics are **reified**: `List<int>` and `List<string>` are different runtime types, `typeof(T)` works inside a generic method, and `List<int>` stores real `int`s (no boxing).

Part 3. Exceptions

Feature	C#	JS
Catch by type	`catch (NotFoundException ex) { }` then `catch (Exception ex) { }`	one `catch (e)` + `instanceof` checks
Catch order	most specific **first**; general before specific = **compile error**	n/a
Filters	`catch (NotFoundException ex) when (ex.Key is int id && id > 10)`	n/a
Rethrow	`throw;` keeps the original stack trace	`throw e`
What can be thrown	only `Exception` objects (or subclasses)	anything, even strings
`finally`	always runs: success, caught, or still flying	same

3.1 throw; vs throw ex;

Demo section 6 output (shortened):

```
throw;    -> top frame: ThrowDeepInside()   <- where the error REALLY happened
throw ex; -> top frame: Rethrow(...)        <- original location LOST
```

`throw ex;` **resets the stack trace** to the current line, so logs and debugging lose where the bug was. Always rethrow with `throw;` (compiler warning CA2200). Names like `<<Main>$>g__ThrowDeepInside|0_6` are compiler-generated names for local functions: read the method name inside.

3.2 Expected vs unexpected errors

	Expected error	Unexpected error (bug / infrastructure)
ProjectHub examples	duplicate project name, empty title, project id does not exist	`SingleOrDefault` with two projects sharing an Id (`InvalidOperationException`); `Create(null!, ...)` -> `NullReferenceException` on `Trim`; `Max()` on empty without `DefaultIfEmpty`; database down (later)
Whose fault	the **caller's input**	**our code** or the environment
What to do	catch where you can turn it into an answer for the user: HTTP **4xx**	do **not** catch locally; a global handler logs it and returns HTTP **500** (topic 12)
Type	`BusinessRuleException`, `NotFoundException`	whatever .NET throws

- **Catch only what you can handle.** Catching `Exception` everywhere hides bugs; never write an empty `catch { }`.
- **Do not use exceptions for normal control flow.** If 'not found' is ordinary, return `null` or use the **Try pattern** (`TryGetValue`, `int.TryParse`).
- **Custom types carry meaning and data**: catch by type; map type -> HTTP status; `ex.EntityName`, `ex.Key`.
- `finally` / `using var x = ...;` for cleanup (files, DB connections).

3.3 null vs exception: query vs command

Method	Behaviour	Design reason
`GetById(id)`	returns `Project?` (null when missing)	a **query**: 'is there a project 99?' 'No' is a valid answer; the caller decides
`AddTask(projectId, ...)`	throws `NotFoundException`	a **command** that **cannot be carried out**; returning null would let the caller continue as if it worked

Part 4. Our code, explained

4.1 The exceptions

src/ProjectHub.Api/Exceptions/NotFoundException.cs

```
namespace ProjectHub.Api.Exceptions;

// Thrown when a requested entity does not exist. Topic 12 maps it to HTTP 404.
public class NotFoundException(string entityName, object key)
    : Exception($"{entityName} with id '{key}' was not found.")
{
    public string EntityName { get; } = entityName;
    public object Key { get; } = key;
}
```

src/ProjectHub.Api/Exceptions/BusinessRuleException.cs

```
namespace ProjectHub.Api.Exceptions;

// Thrown when a request breaks a business rule (an EXPECTED error, caused by the caller's input).
// Topic 12 maps it to an HTTP 4xx response.
public class BusinessRuleException(string message) : Exception(message);
```

- `: Exception(...)`: inherits from `System.Exception` and passes the message to the base constructor, like `super(message)` in `class MyError extends Error`.
- **Primary constructor** `class NotFoundException(string entityName, object key)`: parameters on the class line (C# 12+), short for a constructor that receives them. `EntityName { get; } = entityName` stores one in a read-only property.
- `object key`: `object` is the base type of everything (like `unknown`): ids may be `int` today and `Guid` later.
- Not named `ValidationException` because `System.ComponentModel.DataAnnotations.ValidationException` already exists and ASP.NET uses it.

4.2 The contract: IProjectService

src/ProjectHub.Api/Services/IProjectService.cs

```
using ProjectHub.Api.Entities;

namespace ProjectHub.Api.Services;

public interface IProjectService
{
    IReadOnlyList<Project> GetAll();

    // Returns null when the project does not exist: "not found" is a normal answer here.
    Project? GetById(int id);

    // Throws BusinessRuleException when the name is empty, too long, or already used.
    Project Create(string name, string? description);

    // Throws NotFoundException when the project does not exist.
    TaskItem AddTask(int projectId, string title);
}
```

Return types are part of the contract: `IReadOnlyList<Project>` = 'read, do not change the list'; `Project?` = 'may be null, check it'. C# has no `throws` clause (unlike Java), so comments document which exceptions each method throws.

4.3 The implementation: ProjectService

src/ProjectHub.Api/Services/ProjectService.cs

```
using ProjectHub.Api.Entities;
using ProjectHub.Api.Exceptions;

namespace ProjectHub.Api.Services;

// In-memory implementation. Topic 10 replaces the List with EF Core + SQL Server.
// Not thread-safe yet: fine for a single caller, revisited when it becomes a shared service (Topics 6-7).
public class ProjectService : IProjectService
{
    public const int MaxNameLength = 100;

    private readonly List<Project> _projects;
    private int _nextProjectId;
    private int _nextTaskId;

    public ProjectService(IEnumerable<Project> initialProjects)
    {
        _projects = initialProjects.ToList();

        // DefaultIfEmpty(0): Max() on an empty sequence would throw (Topic 2 gotcha).
        _nextProjectId = _projects.Select(p => p.Id).DefaultIfEmpty(0).Max() + 1;
        _nextTaskId = _projects.SelectMany(p => p.Tasks).Select(t => t.Id).DefaultIfEmpty(0).Max() + 1;
    }

    public IReadOnlyList<Project> GetAll() => _projects.AsReadOnly();

    public Project? GetById(int id) => _projects.SingleOrDefault(p => p.Id == id);

    public Project Create(string name, string? description)
    {
        var trimmedName = name.Trim();

        if (trimmedName.Length == 0)
        {
            throw new BusinessRuleException("Project name is required.");
        }

        if (trimmedName.Length > MaxNameLength)
        {
            throw new BusinessRuleException($"Project name must be at most {MaxNameLength} characters.");
        }

        // The business rule: project names are unique, ignoring case ("Website" == "WEBSITE").
        if (_projects.Any(p => string.Equals(p.Name, trimmedName, StringComparison.OrdinalIgnoreCase)))
        {
            throw new BusinessRuleException($"A project named '{trimmedName}' already exists.");
        }

        var project = new Project
        {
            Id = _nextProjectId++,
            Name = trimmedName,
            Description = description,
        };
        _projects.Add(project);
```

(continued)

```
        return project;
    }

    public TaskItem AddTask(int projectId, string title)
    {
        var project = GetById(projectId) ?? throw new NotFoundException(nameof(Project), projectId);

        var trimmedTitle = title.Trim();
        if (trimmedTitle.Length == 0)
        {
            throw new BusinessRuleException("Task title is required.");
        }

        var task = new TaskItem
        {
            Id = _nextTaskId++,
            ProjectId = project.Id,
            Title = trimmedTitle,
        };
        project.Tasks.Add(task);
        return task;
    }
}
```

4.4 The constructor of ProjectService (your request)

The constructor

This block is the **constructor** of `public class ProjectService : IProjectService`:

```
public ProjectService(IEnumerable<Project> initialProjects)   // <- CONSTRUCTOR
{
    _projects = initialProjects.ToList();

    // DefaultIfEmpty(0): Max() on an empty sequence would throw (Topic 2 gotcha).
    _nextProjectId = _projects.Select(p => p.Id).DefaultIfEmpty(0).Max() + 1;
    _nextTaskId = _projects.SelectMany(p => p.Tasks).Select(t => t.Id).DefaultIfEmpty(0).Max() + 1;
}
```

How to recognize it: same name as the class and **no return type** (no `void`, no `Project`). Only a constructor looks like that.
When it runs: once per object, on `new ProjectService(data)`. Its job is to set up the starting state: copy the data, compute the next ids.
TS equivalent: `constructor(initialProjects: Iterable<Project>) { this.projects = [...initialProjects]; ... }`.
C#-specific: `readonly` fields (`_projects`) can be assigned **only here** (or in their declaration); after that they are locked.
Two constructor styles you have seen: classic (this one: a member inside the class body) and **primary** (the exceptions: `class NotFoundException(string entityName, object key)`). Same concept, different syntax.

Why the data comes in through the constructor (constructor injection)

- If the service called `SampleData.CreateProjects()` itself, the dependency would be **hidden and hard-coded**.
- Passing it in: **tests choose the data** (empty list, a project named X then create X again), and the **source is swappable** without editing the service.
- This is **dependency injection** in its simplest form; topic 7 lets ASP.NET do it automatically.
- TS analogy: a class that `import`s a module singleton inside itself (testing needs `jest.mock`) vs a class that receives the dependency in its constructor (pass a fake).

4.5 Line by line

Line	Why
`public class ProjectService : IProjectService`	implements the contract; compiler checks every member
`private readonly List<Project> _projects`	`readonly`: the field cannot be reassigned after the constructor (list contents still can). `_camelCase` = private field
`_projects = initialProjects.ToList()`	own copy; enumerate once (topic 2); caller's later changes do not affect it
`.Select(p => p.Id).DefaultIfEmpty(0).Max() + 1`	next id; `DefaultIfEmpty(0)` avoids Max-on-empty (topic 2)
`public const int MaxNameLength = 100;`	compile-time constant, reusable by the API layer later
`GetAll() => _projects.AsReadOnly()`	read-only wrapper; cast + `Add` throws `NotSupportedException` (demo 8)
`GetById(...) => _projects.SingleOrDefault(...)`	`Single`: ids must be unique; a duplicate is a bug worth surfacing
`name.Trim()`	`" Docs portal "` and `"Docs portal"` are the same name
three `if ... throw new BusinessRuleException(...)`	**guard clauses**: check and exit early; the happy path stays flat
`string.Equals(a, b, StringComparison.OrdinalIgnoreCase)`	**the business rule** (unique names, ignoring case). Better than `ToLower()`: no new strings, no dependence on server **culture** (Turkish 'i')
`Id = _nextProjectId++`	use the current value, then increment
`GetById(projectId) ?? throw new NotFoundException(...)`	**throw expression**: get it or throw
`nameof(Project)`	the string "Project", checked by the compiler; follows renames

4.6 Known limitations (on purpose)

1. **Not thread-safe.** Two simultaneous requests could get the same id or corrupt the list. Addressed when the service is shared across requests (topics 6-7); EF Core makes the database handle ids (topic 10).
2. **Entities leak.** `GetAll()` is read-only as a list, but the `Project` objects inside are mutable: demo section 8 renames a project from outside. Topic 6 fixes it with **DTOs** (the API returns copies, never entities).

Part 5. var vs an explicit (interface) type (your question)

You asked: in `IReadOnlyList<Project> all = service.GetAll();` can I write `var all = service.GetAll();`? You tried it and it works, and you prefer `var`.

- **Here it is identical.** `GetAll()` already returns `IReadOnlyList<Project>`, so `var` infers exactly that (hover in Rider to see it). The explicit type in demo section 8 was for teaching: the section is *about* the type.
- **Where it is NOT just style** (demo section 1):

```
IProjectService service = new ProjectService(data);   // type: IProjectService (only the contract is visible)
var service             = new ProjectService(data);   // type: ProjectService  (the concrete class)
```

`var` infers from the right side, which is the concrete class. Writing the interface type is a deliberate choice: 'this code depends only on the contract'. In real ASP.NET code you rarely write this line, because DI (topic 7) hands you an `IProjectService`.

Remember

Rule: `var` when the right side already has the type you want; an explicit type when you want a **different, usually more general** type (the interface instead of the class). Preferring `var` is a fine habit.

Part 6. The demo: playground/03-services-exceptions.cs

Run from the repo root: `dotnet run playground/03-services-exceptions.cs`. It is also listed in the `playground` solution folder in Rider.

#	Section	Shows
1	Program against the interface	variable typed `IProjectService`; `GetById(99)` returns null
2	Happy path	`Create` trims the name; `AddTask` gets default status `Todo`
3	The business rule	duplicate (different case), empty, too long -> `BusinessRuleException`
4	NotFoundException	carries `EntityName` and `Key`, not only a message
5	catch order, `when`, `finally`	specific before general; filter; finally always runs
6	`throw;` vs `throw ex;`	which stack trace survives
7	Polymorphism + decorator	`LoggingProjectService` wraps any `IProjectService`; calling code unchanged
8	Encapsulation	read-only list (cast + Add throws) vs mutable entities (the leak)
9	Generic method + constraint	`Largest<T>` with int, string, DateTime
10	Generic type	`Page<T>` for strings and tasks

Full output (captured in the cloud container)

```
=== 1. Program against the INTERFACE, not the class ===
GetAll: 3 projects
GetById(1):  Website redesign
GetById(99): null (not found is a normal answer)

=== 2. Happy path: Create + AddTask ===
Created #4 'Docs portal' (name was trimmed)
Added task #8 'Write getting started' to project #4, status Todo

=== 3. The business rule: expected errors -> BusinessRuleException ===
Create('WEBSITE REDESIGN'): rejected -> A project named 'WEBSITE REDESIGN' already exists.
Create('   '): rejected -> Project name is required.
Create(xxxxxxxxxxxxxxxxxxxx...): rejected -> Project name must be at most 100 characters.

=== 4. NotFoundException carries data, not only a message ===
Message:    Project with id '42' was not found.
EntityName: Project, Key: 42

=== 5. catch order, 'when' filters, finally ===
AddTask(1, "") ...
  caught BusinessRuleException: Task title is required.
  finally: runs no matter what
AddTask(42, "") ...
  caught NotFoundException for a big id: Project with id '42' was not found.
  finally: runs no matter what

=== 6. throw; vs throw ex;  (stack trace) ===
throw;    -> top frame: at Program.<<Main>$>g__ThrowDeepInside|0_6()
throw ex; -> top frame: at Program.<<Main>$>g__Rethrow|0_5(Boolean useThrowEx)

=== 7. Polymorphism: a different implementation behind the same interface ===
  [log] GetAll()
PrintProjectCount: 3 projects
  [log] Create('Mobile app')
  [log] Create failed: A project named 'Mobile app' already exists.
Create('Mobile app'): rejected -> A project named 'Mobile app' already exists.

=== 8. Encapsulation: GetAll() returns a READ-ONLY list ===
Cast + Add: NotSupportedException: Collection is read-only.
service.GetById(1).Name = Renamed by a caller!   <- leak (DTOs fix this in Topic 6)

=== 9. Generics: our own generic method with a constraint ===
Largest(int):      9
Largest(string):   zebra
```

```
Largest(DateTime): 2026-10-14

=== 10. Generics: our own generic type ===
Page<string>:   page 1, 2 of 4: Renamed by a caller!, Mobile app
Page<TaskItem>: page 2, 3 of 8: #4, #5, #6
```

Note: section 9's `DateTime` result and the 'Renamed by a caller!' name in section 10 come from earlier sections changing data (section 2 created a project today; section 8 renamed project 1).

Part 7. Debugging: stop where an exception is thrown

A caught exception is invisible: the program just continues. A **.NET exception breakpoint** makes Rider stop at the `throw` line anyway.

7.1 Steps

1. Paste the snippet below into `Program.cs` right before `app.Run();`.
2. Debug once with no breakpoints: it does not stop (the exception is caught; only the console line appears).
3. Add an exception breakpoint: **Run > Stop on Exception...**, add `BusinessRuleException`, OK. (Other Rider versions: Run > View Breakpoints > + > .NET Exception Breakpoint.)
4. Debug again: Rider stops **inside** `ProjectService.Create`**, on the** `throw` **line**, although the exception will be caught.
5. Inspect the Variables panel (and Frames).
6. Break it: remove the `try/catch` and debug again.
7. Clean up: delete the snippet and the exception breakpoint.

```
var debugService = new ProjectHub.Api.Services.ProjectService(ProjectHub.Api.Data.SampleData.CreateProjects());
try
{
    debugService.Create("Mobile app", null);   // duplicate name
}
catch (ProjectHub.Api.Exceptions.BusinessRuleException ex)
{
    Console.WriteLine($"Caught: {ex.Message}");
}
```

7.2 What you saw

Question	Result
Where did Rider stop?	on `throw new BusinessRuleException($"A project named '{trimmedName}' already exists.");` (correct). Exception breakpoints stop when an exception is **thrown**, not when it is caught: the tool for 'where does this error come from?'
`trimmedName`?	`"Mobile app"` (correct)
After removing try/catch?	you said 'compile failed'. **Wrong term**, see 7.4

7.3 Reading the Variables panel (your screenshot)

Entry	What it is
`$exception`	Rider **pseudo-variable**: the exception being thrown; expand for `Message`, `StackTrace`
`this = ProjectService`	the **object whose method is running**; expand to see private fields (`_projects`, `_nextProjectId`)
`description`, `name` (icon with parentheses)	the method's **parameters**: values passed in
`trimmedName`, `project`	**local variables**

Entry	What it is
`project = null`	not assigned yet: the `throw` happened before `var project = new Project {...}` ran (like `count = 0` in topic 1)

Frames (next to Variables) would show `Create` at the top and your `Program.cs` line below it, as in topic 2.

7.4 Compile time vs runtime: what really happened

Correction

'Compile failed' means the **build** broke and the program never started. Here the build **succeeded**; the program ran and **crashed at runtime**.

Without `try/catch` the exception is **unhandled**: .NET stops the process and prints `Unhandled exception. ProjectHub.Api.Exceptions.BusinessRuleException: A project named 'Mobile app' already exists.` The snippet ran before `app.Run()`, so the API never started.

Later, inside an HTTP request, an unhandled exception does **not** kill the server: ASP.NET catches it for that request and returns **500**; topic 12 turns that into proper 404/400 responses.

Part 8. Your answers, corrections, interview Q&A

8.1 CHECK round 1: what to improve

Question	Your answer	Complete answer
Interface vs class + two benefits	contract vs implementation; interface-typed variable sees only the contract (good). **No benefits given**	swap implementations (EF in topic 10), fakes in tests, DI (topic 7), decorators
How a class matches an interface	declares it; must have all methods (correct)	plus: the compiler checks it; interfaces exist at runtime
Interface vs abstract class	contract vs shared helper code (roughly right)	abstract class can hold state and partial implementation; one base class vs many interfaces
`where T : IComparable<T>`	can call CompareTo; Project -> compile error (correct)	
`throw;` vs `throw ex;`	correct	
Expected vs unexpected	NotFoundException as expected (correct); **no unexpected example; skipped where to catch**	unexpected: duplicate Ids in SingleOrDefault, null name -> NullReferenceException; expected -> 4xx via handler; unexpected -> not caught locally, logged, 500
null vs throw	**mechanics** (SingleOrDefault, `?? throw`)	design: query (null is a valid answer) vs command that cannot be done
Why custom exceptions	**'reusability'**	catch by type (expected vs bug), map type -> HTTP status, structured data
Why data via constructor	**did not know**	no hidden hard-coded dependency, tests choose data, swappable source: constructor injection
OrdinalIgnoreCase vs ToLower	Turkish 'i' (correct)	also: ToLower allocates new strings; ordinal = raw characters, fast and predictable

8.2 Re-check (all correct)

Question	Your answer
null vs throw	GetById null is expected ('no project with this id'); AddTask cannot add a task to a non-existing project. Interview wording: query vs command that cannot be carried out.
Custom exceptions	catch by type; structured data. Add: so a global handler maps each type to an HTTP status.
Data via constructor	no hidden dependency; swappable source; simple dependency injection. Add: tests pass their own data.

8.3 Interview questions with model answers

Q: Interface vs class?

An interface is a contract (members without implementation); a class is an implementation. Code depends on the interface so implementations can be swapped (in-memory -> EF), faked in tests, wired by DI, or decorated.

Q: How does a C# class 'match' an interface, compared to TS?

Nominally: it must declare `: IProjectService`, and the compiler checks every member. TS matches by shape; C# interfaces also exist at runtime.

Q: Interface vs abstract class?

Interface: contract only, a class can implement many. Abstract class: can have state and partial implementation, only one base class. Interface for contracts/capabilities, abstract class for shared code among closely related types.

Q: What is polymorphism? Example?

One interface, many implementations chosen at runtime: `LoggingProjectService` and `ProjectService` both are `IProjectService`; callers do not change.

Q: What does `where T : IComparable<T>` do?

Constrains T so CompareTo can be called; types without IComparable<T> (e.g. Project) are rejected at compile time.

Q: How are C# generics different from TS generics?

Reified: they exist at runtime (`List<int>` vs `List<string>` are different types, `typeof(T)` works). TS generics are erased.

Q: `throw;` vs `throw ex;`?

`throw;` rethrows preserving the original stack trace; `throw ex;` resets it to the current line, losing where the error happened.

Q: Expected vs unexpected errors?

Expected: caused by input (duplicate name, missing project) -> custom exceptions -> HTTP 4xx. Unexpected: bugs/infrastructure -> not caught locally -> global handler logs and returns 500.

Q: When return null and when throw?

Return null (or use Try pattern) when absence is a normal answer to a query; throw when a command cannot be carried out.

Q: Why custom exception types?

Catch by type, map type to HTTP status without parsing messages, carry structured data.

Q: Why should a service get its dependencies via the constructor?

Constructor injection: no hidden hard-coded dependencies; tests pass their own data/fakes; implementations are swappable; ASP.NET DI automates it.

Q: Why `StringComparison.OrdinalIgnoreCase` instead of `ToLower()`?

No string allocations, no culture dependence (Turkish i), predictable for identifiers.

Q: What is a constructor? How do you recognize it?

A member with the class's name and no return type that runs once per `new`; the only place `readonly` fields can be assigned. Primary constructors put parameters on the class line.

Q: What does `finally` guarantee?

It runs after success, after a caught exception, and when the exception keeps propagating; used for cleanup (or `using`).

Q: Can you put `catch (Exception)` before `catch (NotFoundException)`?

No: compile error, the later catch would be unreachable. Most specific first.

Q: What is an exception breakpoint?

A debugger breakpoint that stops when a given exception type is thrown, even if it is caught later.

Q: Compile error vs runtime exception?

Compile error: the build fails, nothing runs. Runtime exception: the program runs and fails while executing; unhandled -> the process (or the request) fails.

Q: var vs explicit type: when does it matter?

When you want a different type than the right side: `IProjectService s = new ProjectService(...)` depends on the contract; `var` would be the concrete class.

Part 9. Summary

Key takeaways

Interface = contract, **class** = implementation. C# matches interfaces **nominally** (`: IProjectService`). Program against the interface: swap, test, DI, decorate.
Abstract class: state + partial implementation, one base class. **Interfaces**: many per class. Prefer composition over inheritance.
Generics: type parameters with **constraints** (`where T : ...`); C# generics are **reified** (exist at runtime).
Exceptions: catch by type, most specific first, `when` filters, `finally` always runs. `throw;` never `throw ex;`.
Expected errors (input) -> custom exceptions -> 4xx. **Unexpected** errors (bugs) -> not caught locally -> logged 500.
Query returns null, command throws. Use the Try pattern instead of exceptions for normal 'not found'.
ProjectService: constructor injection of data, guard clauses, unique-name business rule with `OrdinalIgnoreCase`, `?? throw`, `nameof`, `AsReadOnly`.
Constructor: same name as the class, no return type, runs once per `new`; primary constructors put parameters on the class line.
Debugging: .NET exception breakpoints stop at the `throw`; Variables shows `$exception`, `this`, parameters, locals. Unhandled exception = runtime crash, not a compile error.

Cheat sheet

TS / JS	C#
`interface IFoo { bar(): void }` + `class Foo implements IFoo`	`public interface IFoo { void Bar(); }` + `public class Foo : IFoo`
structural match	nominal: must declare `: IFoo`
`abstract class Base`	`public abstract class Base`
`function f<T extends Comparable<T>>(x: T[])`	`static T F<T>(IEnumerable<T> x) where T : IComparable<T>`
`class MyError extends Error { constructor(m) { super(m) } }`	`public class MyError(string m) : Exception(m);`
`catch (e) { if (e instanceof X) ... }`	`catch (X ex) { } catch (Exception ex) { }`
`throw e` (rethrow)	`throw;`
`constructor(deps) { this.deps = deps }`	`public Service(IDeps deps) { _deps = deps; }`
`x ?? throwError()`	`x ?? throw new NotFoundException(...)`
`a.toLowerCase() === b.toLowerCase()`	`string.Equals(a, b, StringComparison.OrdinalIgnoreCase)`

Checklist: can you do this without notes?

- Explain interface vs class vs abstract class, with two concrete benefits of interfaces.
- Explain why `IProjectService service = new ProjectService(...)` differs from `var service = new ProjectService(...)`.
- Read a generic method with a constraint and say what it allows and rejects.
- Classify an error as expected or unexpected, with a ProjectHub example, and say where it is handled.
- Explain query-returns-null vs command-throws.
- Explain `throw;` vs `throw ex;`.

- Explain constructor injection and why `ProjectService` receives its data.
- Point at the constructor in `ProjectService` and explain each line.
- Use a .NET exception breakpoint and read the Variables panel.
- Tell a compile error from a runtime exception.

What comes next: Topic 4 (async/await)

`Task` and `Task<T>` are C#'s `Promise`; `async`/`await` look almost the same as in TS. The differences that matter on a backend: threads and the thread pool, why I/O must be awaited (scalability), `CancellationToken` (like `AbortController`), and pitfalls (`.Result`, `async void`). Build: make `ProjectService` async and pass a `CancellationToken` through one path.